NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 26, 2026*

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça
Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte
Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 26, 2026*

Abstract

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis LaTeX (novathesis) (LaTeX) template will respect them. Otherwise, the default rule in the novathesis template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more · The last keyword

Resumo

Independentemente da língua em que a dissertação está escrita, geralmente esta contém pelo menos dois resumos: um resumo na mesma língua do texto principal e outro resumo numa outra língua.

A ordem dos resumos varia de acordo com a escola. Se a sua escola tiver regulamentos específicos sobre a ordem dos resumos, o template (LaTeX) novathesis irá respeitá-los. Caso contrário, a regra padrão no template novathesis é ter em primeiro lugar o resumo *no mesmo idioma do texto principal* e depois o resumo *no outro idioma*. Por exemplo, se a dissertação for escrita em português, a ordem dos resumos será primeiro o português e depois o inglês, seguido do texto principal em português. Se a dissertação for escrita em inglês, a ordem dos resumos será primeiro em inglês e depois em português, seguida do texto principal em inglês. No entanto, esse pedido pode ser personalizado adicionando um dos seguintes ao arquivo `5_packages.tex`.

```
\abstractorder(<MAIN_LANG>):={<LANG_1>,...,<LANG_N>}
```

Por exemplo, para um documento escrito em Alemão com resumos em Alemão, Inglês e Italiano (por esta ordem), pode usar-se:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave · Outra palavra-chave · Mais uma palavra-chave · A última palavra-chave

Contents

List of Figures

List of Tables

Acronyms

ACID Atomicity, Consistency, Isolation, Durability *(pp. 7, 8)*
AI Artificial Intelligence *(pp. 4, 5, 10, 11)*
API application programming interface *(pp. 5–7, 9, 11)*

BaaS Backend-as-a-Service *(pp. v, 4–6)*
BASE Basically Available, Soft State, Eventually Consistent *(pp. 7, 9)*
BPMN Business Process Model and Notation *(p. 3)*

CRUD Create, Read, Update, Delete *(p. 6)*
CSS Cascading Style Sheets *(p. 11)*

DI Department of Computer Science *(pp. 1, 3, 5, 8, 9)*

EJS Embedded JavaScript *(p. 7)*
ERD Entitiy-Relationships Diagrams *(p. 3)*

FCT NOVA School of Science and Technology *(pp. 1, 3, 5)*

HTML HyperText Markup Language *(pp. 7, 11, 12)*

I/O Input/Output *(p. 7)*
ISR Incremental Static Regeneration *(p. 11)*

JDBC Java Database Connectivity *(p. 12)*
JPA Java Persitence API *(pp. 6, 9)*
JSON JavaScript Object Notation *(pp. 8, 9)*
JSONB JavaScript Object Notation Binary *(p. 8)*
JSX JavaScriptXML *(p. 10)*

LLM large language model *(pp. 5, 13)*

NoSQL Not only SQL *(pp. v, 4, 7–9)*
NOVA NOVA University Lisbon *(pp. 1, 3, 5)*
novathesis NOVAthesis LaTeX *(pp. i, iii)*

RLS Row-Level-Security *(pp. 6, 9)*

SPA Single Page Application *(p. 10)*
SQL Structured Query Language *(pp. v, 4, 6–9)*
SSG Static Site Generation *(p. 11)*
SSR Server-Side-Rendering *(p. 11)*
STOMP Simple Text Oriented Messaging Protocol *(p. 12)*

UI User Interface *(pp. 10–13)*
UX User Experience *(pp. 2, 4, 10)*

WYSIWYG What You See Is What You Get *(p. 12)*

1

Introduction

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a centerpiece of the university experience, essentially enabled through three diferent distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- Undergraduate Internships: Primarly offered by external companies to provide students with their first professional experience.
- Master's Dissertations: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- Engineering Projects: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering priciples to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors overseeing academic quality, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI @NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that inculdes proposal submission, clarification and approval processes, making proposals available to students, report/dissertation submission, and providing documentation to juries. Currently the department relies on a legacy platform to support some of these activities, however this system presents several critical issues that hinder the efficienty of the department.

The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data an increases the administrative burden on coordinators, professors and the secretatriat.
Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- Communication Gaps and Reliability: The current method for "Call for Proposals" is manual, requiring coordinators to send individual emails to companies, advertisting the opportunity send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.

- Inneficient Feedback Loops: Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resbumit proposals based on coordinator comments.

- Stale Company Data: The existing repository contains a large amout of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.

- Manual Protocol Management: For new companies that have never participated in previous editions, the process of signing the required protocols is still enteriely manual. The platform should automate delivery and tracking of these legal documents as soon as a new comapny registers.

- Outdated User Experience (UX): The interface is not intuitive for external stakeholders, making it difficult for comapnies to register, manage their employees accounts, or monitor the status of their proposals in real-time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastrcture, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI@NOVA FCT. The projects aims to transition from a fragmented, manual system to a centralized digital ecosystem.
To achieve this goal, the following specific objectives have been defined for the initial version:

- Process Unification: Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- Workflow Automation: Implement automated procedures for the "Call for Proposals student seriation (ranking), and the validation of proposals.
- Enhanced Communication and Feedback: Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- Document and Protocol Management: Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- Administrative Oversight: Develop advanced administrative tools, including "superuser" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [37] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.
To achieve this goal, the following specific objectives have been defined for the initial version:

1. Requirement Gathering and Analysis: A through analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (Admin, Coordinator, Student, Professor, Company, Secretariat) and their respective use cases.
2. System Modeling and Design: Utilizing Business Process Model and Notation (BPMN) to map complex workflows [25] and Entitiy-Relationships Diagrams (ERD)

to design the database schema [7]. This phase also involves planning the system architecture.

3. AI-Assisted Prototyping (Vibe Coding): The initial implementation phase will leverage Vibe Coding tools, such as Lovable, Figma Make or Google AI Studio, to rapidly generate the frontend and core functional interfaces [9]. These tools excel at creating React-based user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[47]. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.

4. Architectural Transition and Manual Development: Once the prototype reaches a stable state and meets the primary functional requirements, the project will transition from AI-generated code to manual developlment. This move is essential to refine the User Experience, optimize performance, and implement custom features that go beyond the limitations of AI-driven boilerplate.

5. Custom Backend Implementation: To support the department's more complex business logic - such as the specialized student seriation (ranking) algorithms and robust administrative auditing - the system will transition to a dedicated custom backend, likely using Spring Boot [40]. While Supabase is effective for rapid prototyping, a Spring Boot architecture offers superior domain modeling, type safety and maintanability through its layered structure (Controller, Service, Repository). This transition ensures that the platform is not only fast to develop intially but also scalable and reliable for long-term university use.

6. Verification and Validation: The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam.Additionally, the seriation logic will be rigourously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.

7. Documentation and Evaluation: There will be a continuous documentation of the architecture decisions, providing detailed justications for technical choices such as the preference for SQL over NoSQL for structured academic data, and other technical choices.

2

State of the art and Technologies

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient comunication - by systematically evaluating contemporary approaches to backend architecutres, data persitence, frontend frameworks, and AI-assisted development workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as "Vibe Coding" [13] which supports rapid prototyping by allowing developers to steer large language models (LLMs) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are chacterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. In the backend, developers must choose between fully managed BaaS platforms [15] and custom backends that offer fine-grained control over domain logic, performance, and governance . On the frontend, single-page applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

Beyond classical architecutres, AI-assisted developlment tools and practices are reshaping how software is designed and implemented. Under the "Vibe Coding" paradigm, developers describe desired functionalities to an LLM, which generates and refines code iteratively, prioritizing rapid expermentation and high-level intent over manual code authoring. This approach align with agile or small teams to assemble complex prototypes quickly, though it raises questions about maintainability, security, and auditability in production systems. [36]

2.1 Backend Architectures and BaaS

The choice of backend architecture directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student

seriation and multi-role access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [5]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams [1]. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-source BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST [45], significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices. [47]

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control[41], simplifying the enforcement of different permission profiles for students, companies, professors and administrators. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients.[46] For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless "Edge Functions", typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency. [43]

2.1.2 Custom Backend: Spring Boot

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allows developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintanable domain models. [40]

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes [40,

6], which helps track user actions, state transitions, and login-related events - capabilities that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Custom Backend: Express.js (Node.js)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript [12] . Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads [24] such as broadcasting "Call for Proposals" or status updates to large groups of students.

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies [16, 11] for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs [24], helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

2.2 Data Persistence: SQL vs. NoSQL

The selection of a data persitance model is a fundamental architectural decision that dictates how the system ensures data integrity, scalability, and retrieval efficiency [50, 4]. Given the complex ecosystem of the thesis and internships management, which involves multiple stakeholders (Students, Professors, Companies, Secretariat, etc..) and multi-stage lifecycle workflows for academic projects, this section evaluates the trade-offs between Relational (SQL) and Non-Relational (NoSQL) databases.

The choice between SQL and NoSQL has complex implications for institutional systems. Relational databases prioritize data integrity and consistency through Atomicity, Consistency, Isolation, Durability (ACID) compliance [32], while Non-Relational systems typically follow the Basically Available, Soft State, Eventually Consistent (BASE) model, trading immediate consistency for horizontal scalability and schema flexibility [49]. For academic management platforms where data accuracy are indispensable, this distinction is critical.

2.2.1 The Relational Model (SQL)

Relational databases structure data into tables with predefined schemas, utilizing Foreign Keys to establish strict realtionships between entities [17]. This approach provides fundamental guarantees about data quality and consistency that are essential for institutional systems.

- Structured Data and Integrity: The academic context requires rigorous data consistency. For instance, a Candidacy must strictly link to an existing Student and a valid Proposal. The relational model enforces referentials integrity through primary and foregin key constraints, preventing "orphaned" records - a situtation where a child record references a non-existent parent record [38]. This is particularly critical in the DI management platform, where deleted Companies should not retain active Proposals, and Students cannot be assigned to non-existent projects.

- Referential integrity mechanism ensure that every foreign key in a child table corresponds to a valid primary key in the parent table, maintaining logical consistency across the entire database. Beyond preventing data anomalies, referential integrity supports cascading updates and deletions, ensuring that changes propagate correctly across related entities [17, 38].

- Complex Querying: The platform requires complex join operations to generate views - for example, listing all proposals for a specific edition filtered by a company's validation status, or retrieveing all students that are candidates to a particular internship. SQL excels at these complex relationships through its mature and optimzed query language, which can efficiently traverse multiple table relationships in a single query [21].

- PostgreSQL: This project will initially utilize PostgreSQL (via Supabase for rapid prototyping). Beyond standard SQL features, PostgreSQL offers support for complex data types (JSON/JSONB, arrays, enums) and Stored Procedures [26, 30]. This allows the system to handle semi-structured data - such as like flexible feedback forms or email template configurations - while maintaining the benefits of a relational schema for academic entities.

2.2.2 The Non-Relational Model (NoSQL)

NoSQL databases (e.g., document stores like MongoDB or key-value systems like Redis) offer schema flexibility and horizontal scalability, often at the cost of immediate consistency (ACID properties) [50]

- Flexibility vs Structure: While NoSQL allows for rapid iteration of data models without schema migrations, the academic domain is highly structured. Entities such as Users, Editions, Proposals, Companies and Students have well-defined attributes that rarely change dynamically or unexpectedly. The governance requirements of an academic instution demand clear, predictable data structures rather than evolving, loosely-typed records.

- Consistency Challenges: In workflows like Student Seriation, data consistency is essential. A NoSQL appraoch, which often relies on eventual consistency, could introduce race conditions where a student is assigned to a project that is no longer

available or where multiple simulataneous requests lead to conflicting state changes. The BASE model prioritises availability and partition tolerance over strict consistency, which is acceptable for read-heavy analytics systems but unsuitable for transactional workloads that require immediate correctness.

2.2.3 Justification for the Chosen Approach (PostgreSQL)

The decision to utilize PostgreSQL is driven by the specific requirements and constraints of the DI management platform:

1. Data Integrity and Referential Consistency: The need to enforce relationships between Companies, Proposals, Editions, and Students needs a relational schema with foreign key constraints. For examplple, the system must prevent an inactive or deleted company from having active proposals, and it must prevent logically impossible states such as student being assigned to multiple conflicting projects in the same edition. These constraints are best expressed and enforced at the database level through relational integrity mechanisms rather than dispersed throughout the platforms's code [38, 17].

2. Row-Level-Security (RLS) and Multi-Tenancy: Using PostgreSQL enables the implementation of RLS, a database-level access control mechanism. RLS policies allow fine-grained permissions to be defined directly in the database layer, ensuring that access rules (e.g. "a company can only see the candidates to its own propsals", "a student can only see the applications they've made") are enforced uniformly regardless of how the API is accessed. This provides a security layer that cannot be bypassed by the application logic, addressing a critical requirement for institutional data protection, By using PostgreSQL's RLS policies with role-based contexts, the platform achieves secure multi-tenancy at the database level, without relying solely on application-level permission checks. [14]

3. Hybrid Capabilities and Schema Evolution: PostgreSQL provides the strict structure required for academic records while support JSON types, effectively making the for using a NoSQL solution unnecessary. The hybrid approach gives the platform the best of both worlds: schema enforcement where it matters (core academic entities and relationships) and flexibility where needed (dynamic form configurations, email templates, audit logs). This design reduces operational complexity by consolidating storage into a single database system while maintaining both consistency guarantees and the agility to evolve data models as requirements change. [26]

4. Compatibilty and Technology Continuity: Both the rapid protoyping tools (Supabase [42]) and the target production framework (Spring Boot with JPA/Hibernate [39]) are optimized for relational databases, ensuring a smooth transition between development phases. This alignment minimizes architectural friction and ensures

that the knowledge base and tooling remains consistent as the system evolves from prototype to production.

2.3 Frontend Frameworks and User Experience

The "Outdated User Experience (UX)" of the legacy system is identified as a critical failure point, particularly for external stakehoders such as companies who struggle to register, manage accounts, or monitor proposal status. To address this limitations, the new platform must transition from static, server-rendered pages to a modern, reacting Single Page Application (SPA) or hybrid architecture that enables faster navigation, richer interactivity [33], and consistent interfaces across distinct user roles.

2.3.1 React.js: The Foundation for Vibe Coding

React is a free, open-source JavaScript library maintained by Meta that specializes in building interactive user interfaces through component-based architecture [33, 34]. Unlike traditional web development where the entire page must be reloaded on data changes, React efficiently re-renders only the components that have actually changed, using its virtual DOM reconciliation process to minimize performance overhead [22]. React uses JavaScriptXML (JSX), a syntax extension that allows developers to write HTML-like code directly within JavaScript [33], making the UI logic more readable and maintainable.

React is selected as the core library for building the platform's user interface for two primary reasons:

1. Component-Based Architecture: React allows for the creation of reusable, self-contained UI components (e.g. "ProposalCards" where all the details from a proposal are, that can be used in multiple pages) [31]. This modular approach ensures consistency across the distinct interfaces required for Students, Professors, and Comapnies, while enabling efficient code reuse and simplified maintenance.

2. Synergy with AI-Assisted Development: The project methodology relies on Vibe Coding tools for rapid prototyping. Tools such as Lovable, Bolt.new, Figma Make and Google AI Studio are optimized to generate code specifically for the React ecosystem. Using React ensures that the code generated during the prototyping phase is immediately usable and extensible, eliminating the need for time-consuming library migrations.

2.3.2 Next.js: Production-Grade Framework

Next.js is a full-stack React framework created and maintaned by Vercel that extends React's capabilities [23] by providing built-in solutions for routing, data fetching, server-side rendering, and infrastructure - all without complex configuration. While React is a library focused solely on the UI layer and requires additional libraries for routing and server-side

concerns, Next.js provides an integrated toolkit that handles both frontend rendering and backend API logic in a single project [23]. Next.js supports multiple rendering strategies: Server-Side-Rendering (SSR) for dynamic content rendered on-demand, Static Site Generation (SSG) for pre-rendering at build time, and Incremental Static Regeneration (ISR) for updating static content without full rebuilds [20].

Next.js is selected to complement React because it provides essential production-grade features:

- Routing and Performance: Next.js offers a file-system based router (App Router) [3] and rendering strategies such as SSR and SSG. SSR renders pages on demand for dynamic content, while SSG pre-renders static pages at build time, reducing server load times and enabling fast initial page loads [20], which is crucial for retaining the engagement of external company users. This flexibility allows different sections of the platform to use the optimal rendering strategy based on content volatility.

- Scalability and Full-Stack Capabilities: As the platform transitions from a prototype to a long-term solution, Next.js provides the architectural structure needed to manage complex API integrations, and full-stack workflows. In inital phases, Next.js API routes can serve as lightweight backend layer [2] before the full transition to Spring Boot, reducing deployment complexity during the transition period.

2.3.3 Tailwind CSS: Utility-First Styling

Tailwind CSS is a utility-first CSS framework that provides a comprehensive set of low-level, single-purpose utility classes (such as `p-4` for padding, `text-center` for text alignment, or `bg-blue-600` for background color) that can be composed directly in HTML markup without writing custom CSS. Unlike traditional CSS frameworks (such as Bootstrap) that provide pre-built UI components, Tailwind CSS empowers develpers to build custom designs by combining small utility classes, giving them complete design control while maintaining consistency [35].

Tailwind CSS is selected for styling the platform for two key reasons:

- Rapid UI Development: Tailwind utility-first approach enables developers to style elements directly in HTML markup without writing custom CSS. This significantly accelerates the conversion of "mental models" or Figma designs into working code, reducing the time spent switching between markup/code and style files [35].

- AI Integration: Most AI coding assistants (specifically Lovable) default to generating Tailwind CSS code. This ensures that the aesthetic output from the Vibe Coding process is modern, responsive, and easy to maintain without the complexity of managing large external CSS files.

2.3.4 Real-Time Feedback and Interactivity

The legacy system suffers from "Inefficient Feedback Loops" where users must constantly refresh or wait for emails to know the status of a proposal.

- Supabase Realtime (Prototyping Phase): During rapid prototyping with Supabase, Realtime subscriptions powered by WebSockets, allow the frontend to listen to database changes instantly and update the UI components without requiring page refreshes [44]. For example, a student receives an immediate "Selected" when a company confirms an interview result, this improves user engagement.

- Production Architecture: PostgreSQL LISTEN/NOTIFY with Spring Boot WebSocket: When transitioning to Spring Boot, real-time updates leverage PostgreSQL's native LISTEN/NOTIFY mechanism - a lightweight, built-in pub/sub system [29, 28] that requires no external message brokers. The workflow is straightforward.

  1. Database Trigger: When a Proposal status changes (e.g., from "Pending" to "Accepted") a PostgreSQL trigger automatically fires and sends a notification via `pg_notify()` to a named channel [29, 28].
  2. Spring Boot Listener: A background thread in Spring Boot continuously listens on the PostgreSQL notification channel using JDBC's PostgreSQL driver extensions [18]: When a notification is received, it triggers an event in the application.
  3. WebSocket Broadcast: Spring Boot WebSocket infrastructure (optionally using Simple Text Oriented Messaging Protocol (STOMP) for advanced message routing) [48, 40] broadcasts the update to all conecteed frontend clients subscribed to relevant topics. For instance, all students viewing proposals from a specific company receive live updates when that company changes a proposal's status.

2.3.5 Communication Experience (Email UX)

A major pain point is the lack of flexibility in creating email templates and the issue of emails being flagged as spam.

- Template Management: The frontend will feature a What You See Is What You Get (WYSIWYG) markdown or HTML editor [27] (integrated into the Admin Dashboard) allowing coordinators to visually edit email templates, without the knowledge of those markup languages. This eliminates the need for manual template coding and reduces errors.

- Deliverability: To ensure emails reach users inboxes, the system architecture must integrate with an established transaction email service rather than relying on the error-prone, manual email workflows of the legacy system [19]. These services

provide authentication protocols, monitoring, and detailed analytics to maintain sender reputation and maximize inbox placement [10].

2.4 AI-Assisted Development (Vibe Coding)

Traditional software development involves writing code manually, line by line. However, a new paradigm known as "Vibe Coding" has emerged, leveraging large language models (LLMs) to generate full-stack applications or complex User Interface (UI) components through natural language prompting [8, 36]. This section analyzes the capabilities of these tools to determine their viability for the rapid prototyping phase of the dissertation.

2.4.1 Concept and Capabilities

Vibe Coding tools differ from standard code assistants (like Github Copilot) by focusing on generating entire applications or functional modules rather than just autocomplete syntax. They typically integrate a development environment, a live preview window, and an AI agent capable of iterating on code based on user feedback. The primary advantage of this approach is the drastic reduction in time required to move from a conceptual requirement to a testable, interactive interface.

2.4.2 Analysis of Vibe Coding Tools

TODO - need to check the tools better because since I was researching they got updated and some new tools appeared too

2.4.3 Limitations and Architectural Implications

TODO - waiting for the subsection before

3

Requirements and Current System Analysis

3.1 Analysis of the Legacy System

TODO: write a bout the current platform and the specific problems (for example not being used for master's degrees, problems with emails etc)

3.2 Requirementes Gathering

TODO: both functional requirements and non-functional requirements

3.3 Use Cases and Stakeholders

TODO: identify the stakeholders (students, professors, companies, secretariat, coordinators, admins etc) and their interactions

4

Solution Proposal

4.1 Proposed System Architecture

TODO: high-level diagram of the system (interaction frontend, backend, db etc)

4.2 Process Modelling (BPMN)

TODO: mapping of workflows with bpmn

4.3 Data Model (ERD)

TODO: presentations of the ER diagram (db diagram) e definition of the entities there

4.4 API and Component Design

TODO: definition of the main endpoints and the layer logic (controller, service, repository)

4.5 Work Plan

TODO: maybe a cronogram (gantt?) with the development phases, tests, docs writing, risks and mitigations

Bibliography

[1] AlSalah. *Benefits and Drawbacks of Using Backend as a Service*. Point of SaaS. 2025. URL: https://pointofsaas.com/benefits-and-drawbacks-of-using-backend-as-a-service-3/ (cit. on p. 6).

[2] *API Routes*. Next.js Documentation. Vercel. 2025. URL: https://nextjs.org/docs/pages/building-your-application/routing/api-routes (cit. on p. 11).

[3] *App Router (Next.js)*. Next.js Documentation. Vercel. 2026. URL: https://nextjs.org/docs/app (cit. on p. 11).

[4] Ayush Sharma. *Difference between SQL and NoSQL*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/sql/difference-between-sql-and-nosql/ (cit. on p. 7).

[5] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend (cit. on p. 6).

[6] *Chapter 15. Envers*. Hibernate Community. 2013. URL: https://docs.hibernate.org/orm/4.3/devguide/en-US/html/ch15.html (cit. on p. 7).

[7] P. P. Chen. "The Entity-Relationship Model: Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).

[8] Cloudflare, Inc. *What is vibe coding?* 2025. URL: https://www.perplexity.ai/search/add-this-as-a-citation-in-late-dWWatfeETdCTpmIFNr6Jbg?sm=d (cit. on p. 13).

[9] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on p. 4).

[10] David Clayton. *How to Improve Deliverability of Transactional Emails*. SMTP. 2023. URL: https://www.smtp.com/blog/transactional-email/improve-deliverability-of-transactional-emails/ (cit. on p. 13).

[11] *Embedded JavaScript templating Documentation*. EJS. 2026. URL: https://ejs.co/#docs (cit. on p. 7).

[12] *Express.js Documentation*. Express.js Foundation. 2026. URL: https://expressjs.com/ (cit. on p. 7).

[13] Figma. *What is Vibe Coding? Everything You Need to Know*. 2025. URL: https://www.figma.com/resource-library/what-is-vibe-coding/ (cit. on p. 5).

[14] Fred Pham. *Why Your Database Needs Boundaries: An Intro to PostgreSQL's Row Level Security (RLS)*. Shift Asia Dev blog. 2025. URL: https://shiftasia.com/community/why-your-database-needs-boundaries-an-intro-to-postgresqls-row-level-security-rls/ (cit. on p. 9).

[15] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: https://acropolium.com/blog/baas-use-cases/ (cit. on p. 5).

[16] *Handlebars.js Documentation*. Handlebars.js. 2026. URL: https://handlebarsjs.com/ (cit. on p. 7).

[17] IBM Corporation. *Foreign key (referential) constraints*. IBM DB2 12.1.x Documentation. Accessed: 2026-01-21. 2026. URL: https://www.ibm.com/docs/en/db2/12.1.x?topic=constraints-foreign-key-referential (cit. on pp. 7–9).

[18] Jasmin Tankić. *Unlocking the Power of Postgres Listen/Notify: Building a Scalable Solution With Spring Boot Integration*. DZone. 2023. URL: https://dzone.com/articles/leveraging-postgres-listennotify-in-spring-boot (cit. on p. 12).

[19] Ketevan Bostoganashvili and Piotr Malek. *I Compared 7 Best Transactional Email Services: Here's What I Found*. mailtrap. 2025. URL: https://mailtrap.io/blog/transactional-email-services/ (cit. on p. 12).

[20] Mercy Tanga. *SSR vs. SSG in Next.js: Differences, Advantages, and Use Cases*. strapi. 2024. URL: https://strapi.io/blog/ssr-vs-ssg-in-nextjs-differences-advantages-and-use-cases (cit. on p. 11).

[21] Microsoft. *Joins (SQL Server)*. SQL Server Documentation. 2025. URL: https://learn.microsoft.com/en-us/sql/relational-databases/performance/joins?view=sql-server-ver17 (cit. on p. 8).

[22] Nafisa Anjum. *ReactJS Reconciliation*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/reactjs-reconciliation/ (cit. on p. 10).

[23] *Next.js Docs*. Vercel. 2026. URL: https://nextjs.org/docs (cit. on pp. 10, 11).

[24] *Node.js Documentation*. Node.js Foundation. 2026. URL: https://nodejs.org/en/docs/ (cit. on p. 7).

[25] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 3).

[26] Oskar Dudycz. *PostgreSQL JSONB - Powerful Storage for Semi-Structured Data*. 2025. URL: https://www.architecture-weekly.com/p/postgresql-jsonb-powerful-storage (cit. on pp. 8, 9).

[27] Paul Bratslavsky. *10 Best WYSIWYG Editors for 2025*. strapi. 2025. URL: https://strapi.io/blog/best-wysiwyg-editors (cit. on p. 12).

[28] Pedro Alonso. *PostgreSQL LISTEN/NOTIFY: Real-Time Without the Message Broker*. 2025. URL: https://www.pedroalonso.net/blog/postgres-listen-notify-real-time/ (cit. on p. 12).

[29] Philippe Sevestre. *Using PostgreSQL as a Message Broker*. Baeldung. 2023. URL: https://www.baeldung.com/spring-postgresql-message-broker (cit. on p. 12).

[30] PostgreSQL Global Development Group. *8.14. JSON Types*. PostgreSQL 18 Documentation. 2026. URL: https://www.postgresql.org/docs/current/datatype-json.html (cit. on p. 8).

[31] Pranjal Nanda. *React Component Based Architecture*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/react-component-based-architecture/ (cit. on p. 10).

[32] K. Pykes. *What Are ACID Transactions? A Complete Guide for Beginners*. Datacamp. 2025. URL: https://www.datacamp.com/blog/acid-transactions (cit. on p. 7).

[33] React Team. *React*. Official React Documentation. Meta. 2025. URL: https://react.dev/ (cit. on p. 10).

[34] React Team. *React*. Meta Open Source Projects. Meta Open Source. 2026. URL: https://opensource.fb.com/projects/react/ (visited on 2026-01-23) (cit. on p. 10).

[35] Sachin Chaurasiya. *Tailwind CSS: Utility-First Styling for Rapid UI Development*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/blogs/tailwind-css-utility-first-styling-for-rapid-ui-development/ (cit. on p. 11).

[36] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275 (cit. on pp. 5, 13).

[37] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: https://scrumguides.org/ (cit. on p. 3).

[38] R. H. Shaikh. *Referential Integrity: Why It's Vital for Databases*. acceldata. 2024. URL: https://www.acceldata.io/blog/referential-integrity-why-its-vital-for-databases (cit. on pp. 8, 9).

[39] Spring Data JPA Team. *JPA*. Spring Data JPA Reference Documentation. VMware, Inc. 2026. URL: https://docs.spring.io/spring-data/jpa/reference/jpa.html (cit. on p. 9).

[40] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (cit. on pp. 4, 6, 12).

[41] Supabase. *Auth Overview*. 2026. URL: https://supabase.com/docs/guides/auth (cit. on p. 6).

[42] Supabase. *Database*. 2026. URL: https://supabase.com/docs/guides/database/overview (cit. on p. 9).

[43] Supabase. *Edge Functions*. 2026. URL: https://supabase.com/docs/guides/functions (cit. on p. 6).

[44] Supabase. *Realtime*. 2026. URL: https://supabase.com/docs/guides/realtime (cit. on p. 12).

[45] Supabase. *REST API Overview*. 2026. URL: https://supabase.com/docs/guides/api#rest-api-overview (cit. on p. 6).

[46] Supabase. *Row Level Security*. 2026. URL: https://supabase.com/docs/guides/database/postgres/row-level-security (cit. on p. 6).

[47] Supabase. *Supabase: The Open Source Firebase Alternative*. https://github.com/supabase/supabase. Project README. 2019 (cit. on pp. 4, 6).

[48] Tomasz Dąbrowski. *Using Spring Boot for WebSocket Implementation with STOMP*. Toptal. 2026. URL: https://www.toptal.com/developers/java/stomp-spring-boot-websocket (cit. on p. 12).

[49] *What's the Difference Between an ACID and a BASE Database?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-acid-and-base-database/ (cit. on p. 7).

[50] *What's the Difference Between Relational and Non-relational Databases?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-relational-and-non-relational-databases/ (cit. on pp. 7, 8).